

Developer Club wordmark • {{CLUB_LOGO}}

WE START AT

6:55

GET SET UP WHILE YOU EAT

- 1 `git clone https://github.com/joshualent/questions-api.git`
`cd questions-api`
- 2 `uv sync`
- 3 `uv run python manage.py migrate`
- 4 `uv run python manage.py runserver`



1 · Starter repo

QR
appears
when
{{LIVE_B
OARD_URL}}

2 · Ask a question
is set



Done? Open `http://127.0.0.1:8000/admin/`, then scan the second QR code and ask a question.



Developer Club wordmark · {{CLUB_LOGO}}

BACKEND WORKSHOP

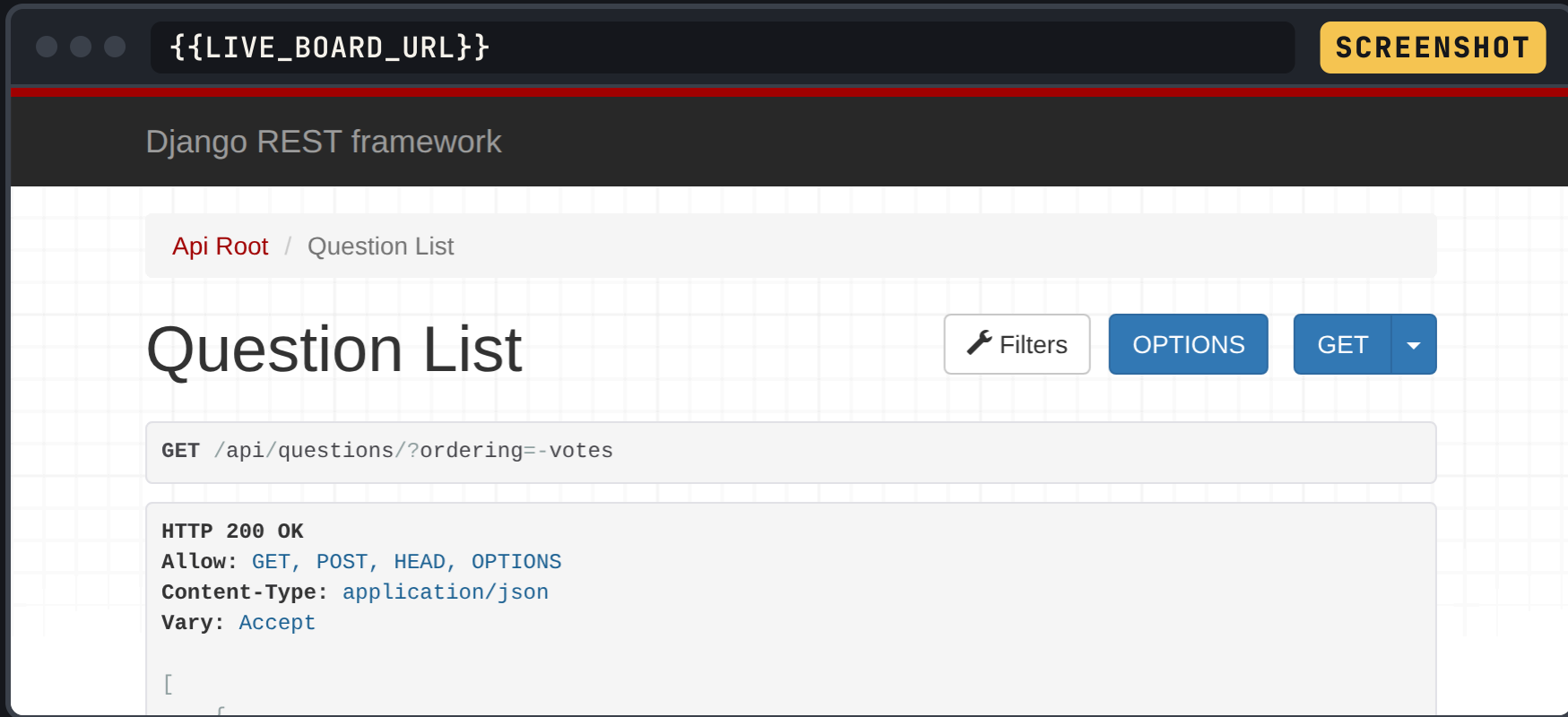
BUILD A Q&A API

with Python, Django & Django REST Framework

Monday, Oct 19 · Floyd Hall D-109

Here's what you'll build

The live board: your questions, sorted by votes.



How tonight works



Show of hands

Python? Django?
Ever called an API?



Code along

Type with me.
Behind? Grab a
checkpoint branch.

```
tier-1-done
```

```
tier-2-auth
```

```
tier-3-m2m
```



Stuck? Hand up.

A helper comes to
you. I keep going.

What is a backend?

The part of an app you never see.



Think of a restaurant

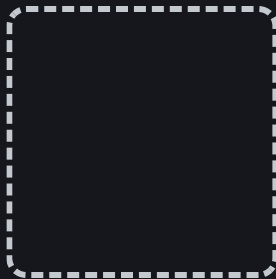
Dining room



your data

= Frontend

Order
window



= API

Kitchen



Order #4
questions, by votes

= Backend

What the backend owns



Store data

Questions and votes, saved for good.



Enforce rules

Text can't be empty. Votes go up by one.



Security

Who can post, vote, or delete.

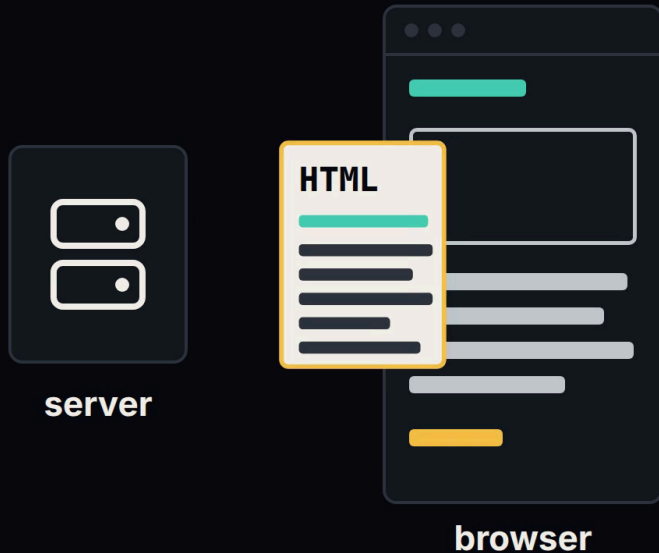


Never trust the client. Anyone can send any request.

Server-rendered pages vs. an API

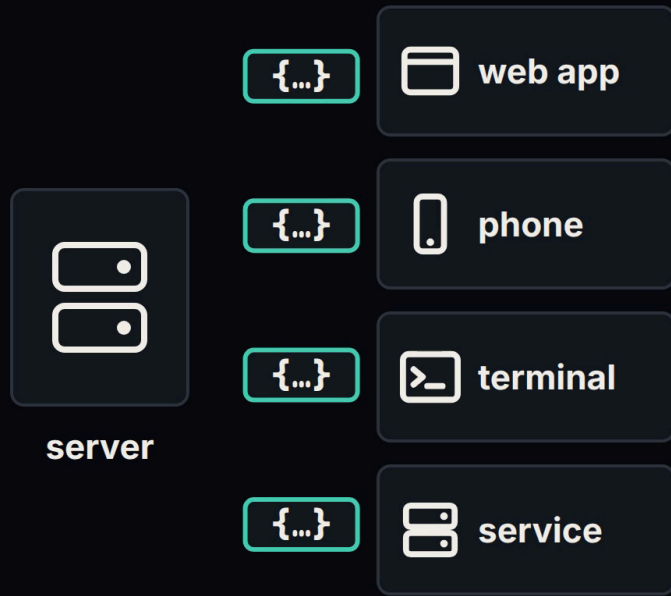
Server-rendered

whole page → one browser



API

small JSON → any client



Why an API?



One backend, many clients

Web, phone, terminal, other services.



Separation of concerns

Frontend and backend change on their own.



Test and scale each side

Independently, as they need it.

Server-rendered apps are still a great choice for many projects. Django does both.

Anatomy of an HTTP request

REQUEST

method **POST** URL `/api/questions/`

headers `Content-Type: application/json`

body `{"text": "What is a serializer?"}`

RESPONSE

status **201 Created**

body `{"id": 7, "text": "What is a serializer?",
"votes": 0, "answered": false,
"created_at": "2026-10-19T19:07:12-04:00"}`

Methods and status codes for tonight

METHODS

GET	read
POST	create, or run an action
PATCH	update (PUT too)
DELETE	delete

STATUS CODES

200 OK	it worked
201 Created	saved a new one
400 Bad Request	invalid data
403 Forbidden	not allowed
404 Not Found	no such thing
500 Server Error	server bug

REST in one slide

Nouns in URLs, verbs in methods.

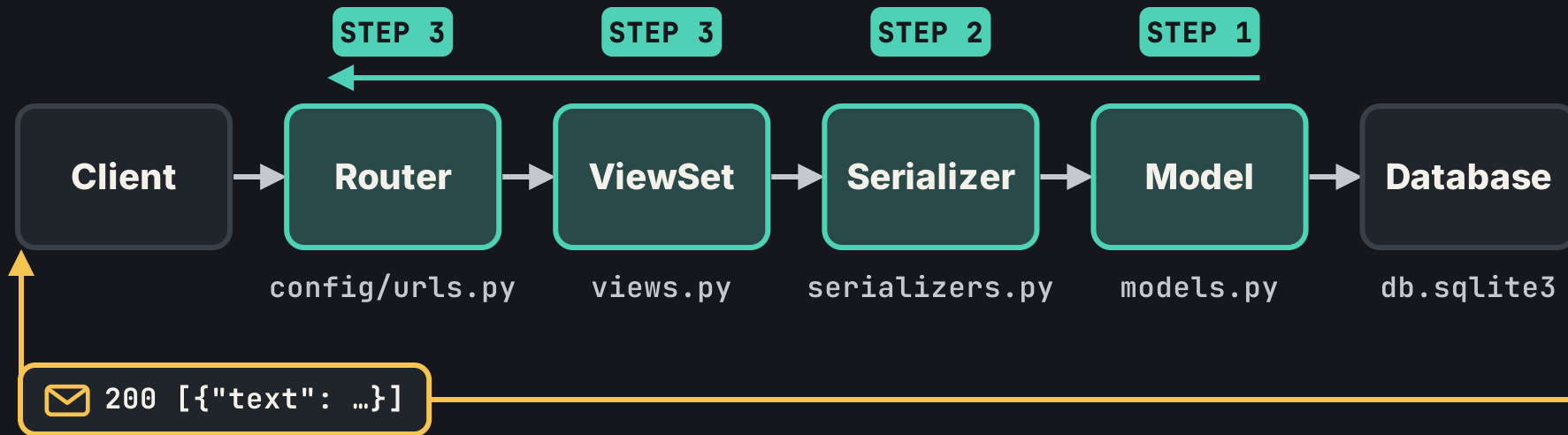
```
/api/questions/5/upvote/
```

POST

an action on one question (step 4)

The map for tonight

Every request takes this path. We build it right to left.



Your starter repo

`questions-api/`

└ `config/`

└ `settings.py` DRF + questions app already wired

└ `urls.py` router, commented out

└ `questions/`

└ `models.py` step 1

└ `serializers.py` step 2

└ `views.py` steps 3–5

└ `migrations/`

└ `manage.py` every command goes through this

You edit the four **highlighted** files, in order. Everything else is ready.

Step 1: the model

```
class Question(models.Model):  
    text = models.CharField(max_length=300)  
    votes = models.IntegerField(default=0)  
    answered = models.BooleanField(default=False)  
    created_at = models.DateTimeField(auto_now_add=True)
```

questions_question

id	text	votes	answered	created_at
1	Is REST hard?	3	False	19:02
2	Do I need SQL?	7	False	19:05
3	How do I deploy?	4	True	19:09

An object is a row

STEP 1 · NOW IN VS CODE

Write the Question model

Client → Router → ViewSet → Serializer → **Model** → Database

FILE

questions/models.py

RUN

```
$ uv run python manage.py makemigrations  
$ uv run python manage.py migrate
```

SEE

Create model Question, then Applying questions.0001_initial... OK

Fields: text votes answered created_at

Step 2: the serializer

Translator (JSON ↔ Python) and bouncer (validation).

JSON IN

SERIALIZER

PYTHON OUT

```
{"text": "What is REST?"}
```

```
Question(text="What is REST?")
```

```
{"text": "Hi", "votes": 9999}
```

read-only: dropped

translator
bouncer

```
Question(text="Hi", votes=0)
```

```
{"text": ""}
```

400

"This field may not be blank."

nothing saved

STEP 2 · NOW IN VS CODE

Write the serializer



FILE

questions/serializers.py

SEE

Nothing to run. No red errors in the runserver terminal.

ASK THE ROOM

What happens if someone sends `"votes": 9999` ?

Step 3: the **ViewSet** and the router

One `ModelViewSet` gives you five endpoints. The router writes their URLs.

`QuestionViewSet`

`list`

`create`

`retrieve`

`update`

`destroy`

`DefaultRouter` writes the URLs

`GET`

`/api/questions/`

`POST`

`/api/questions/`

`GET`

`/api/questions/5/`

`PATCH`

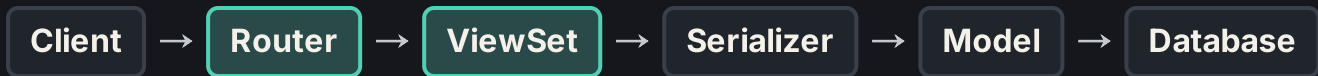
`/api/questions/5/`

`DELETE`

`/api/questions/5/`

STEP 3 · NOW IN VS CODE

ViewSet + router, then test it



FILE

questions/views.py

config/urls.py

SEE

<http://127.0.0.1:8000/api/questions/>

shows the browsable API

POST

{"text": "What is REST?"}

→

201 Created

POST

{"text": ""}

→

400 Bad Request

GET

/api/questions/999/

→

404 Not Found

Step 4: custom actions

Not everything is CRUD. The server decides how votes change.

PATCH /api/questions/5/

{"votes": ~~9999~~}

⊗ votes is read-only: change ignored

POST /api/questions/5/upvote/

no body needed

200 OK the server adds exactly one

question 5

What is REST?



4

votes

Your turn: write `mark_answered`

Add an action that sets `answered` to `True`.

POST

`/api/questions/5/mark_answered/`

HINT

Copy `upvote`, rename it, and change one line.

TWO MINUTES

2:00

Step 5: ordering

Query parameters sort a collection. No new URLs.

GET

127.0.0.1:8000/api/questions/?ordering=-votes|

↑ 9

Is DRF used in industry?

#4

↑ 7

Do I need to know SQL?

#2

↑ 4

How do I deploy this?

#3

↑ 2

Django or Flask for a first project?

#1

STEP 5 · NOW IN VS CODE

Turn on ordering, then commit

Client → Router → ViewSet → Serializer → Model → Database

FILE

questions/views.py

RUN

```
$ git add -A  
$ git commit -m "v1 done"
```

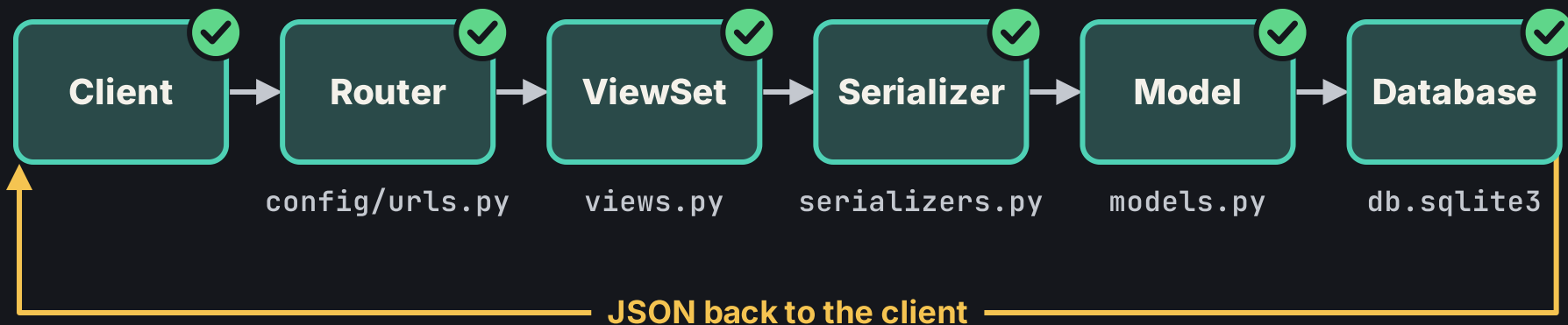
SEE

`/api/questions/?ordering=-votes` lists the most-voted first

```
from rest_framework.filters import OrderingFilter
```

```
filter_backends = [OrderingFilter]  
ordering_fields = ['votes', 'created_at']
```

YOU BUILT A REST API



BEHIND? CATCH UP:

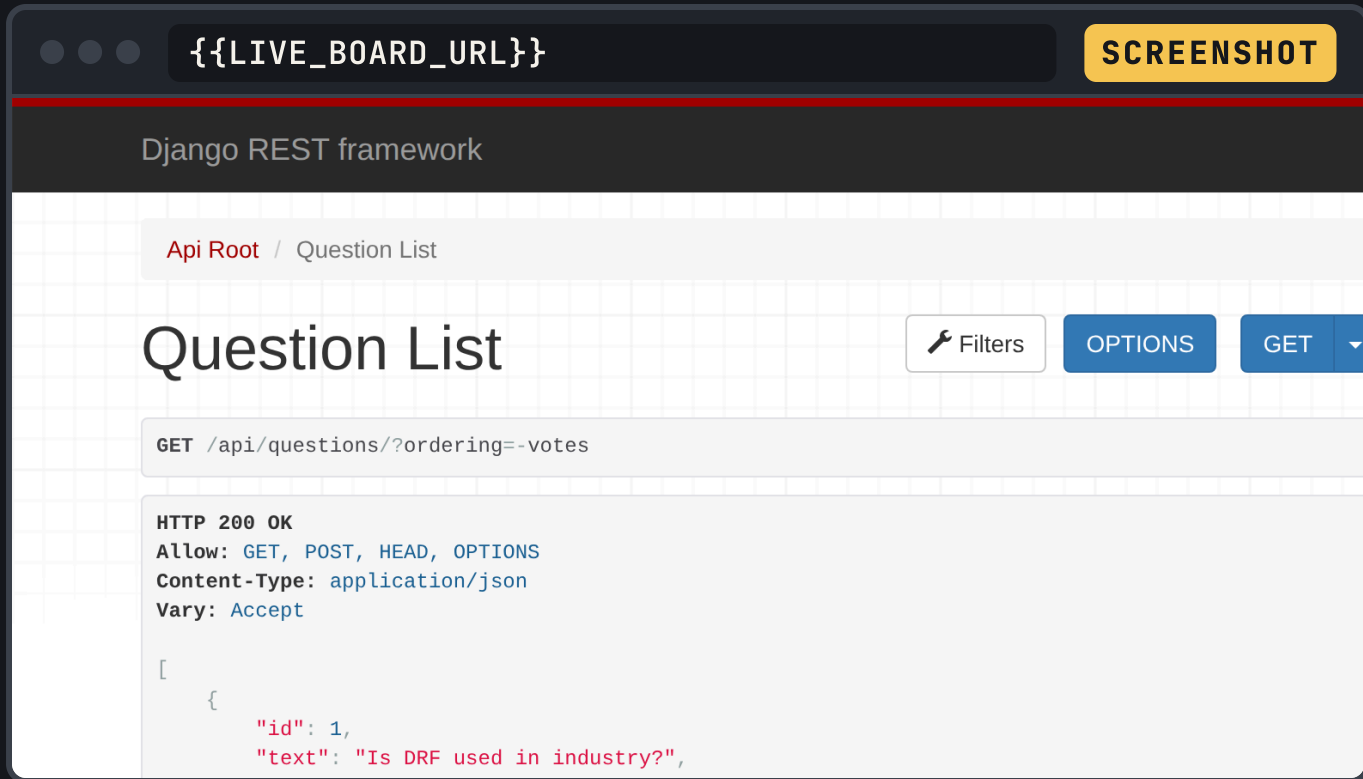
```
git stash -u
```

```
git checkout tier-1-done
```

```
uv run python manage.py migrate
```

Vote now

Upvote what you want answered at the end.



QR appears
when
`{{LIVE_BOARD_URL}}`
is set

Scan to vote

What's wrong with our app?

- One person can upvote forever
- Anyone can mark a question answered
- Anyone can delete any question

🔍 same user, every click

Is pineapple on pizza OK?

⬆ Upvote



9,999

Authentication vs. authorization



Authentication

Who are you?

Logging in.



Authorization

What are you allowed to do?

Permissions.

Tonight: session login. Real apps usually use tokens (JWT), but the permission code stays the same.

A counter can't remember who voted

questions

id	text
1	Is REST hard?
2	Do I need SQL?

questions_question_voters

question_id	user_id
1	1
1	2
1	3
2	1

users

id	username
1	 sam
2	 ana
3	 lee

Django builds the join table

STEP A · STRETCH · VS CODE

Log in, then lock it down

Client → Router → ViewSet → Serializer → Model → Database

FILE

config/urls.py

config/settings.py

RUN

```
$ uv run python manage.py createsuperuser
```

SEE

Uncomment `api-auth/` and `DEFAULT_PERMISSION_CLASSES`



Logged out

POST



403 Forbidden



Logged in

POST



201 Created

Heads up: votes reset

Old votes don't record who cast them.

Is DRF used in industry?

votes



voters

[]

Swapping `votes` for `voters` starts every count at 0.

A real product would write a data migration to keep them.

STEP B · STRETCH · PASTE, DON'T TYPE

One vote per user



FILE

questions/models.py

questions/serializers.py

questions/views.py

RUN

```
$ uv run python manage.py makemigrations
```

```
$ uv run python manage.py migrate
```

SEE

Second upvote by the same user → 400 Bad Request

PROVE IT

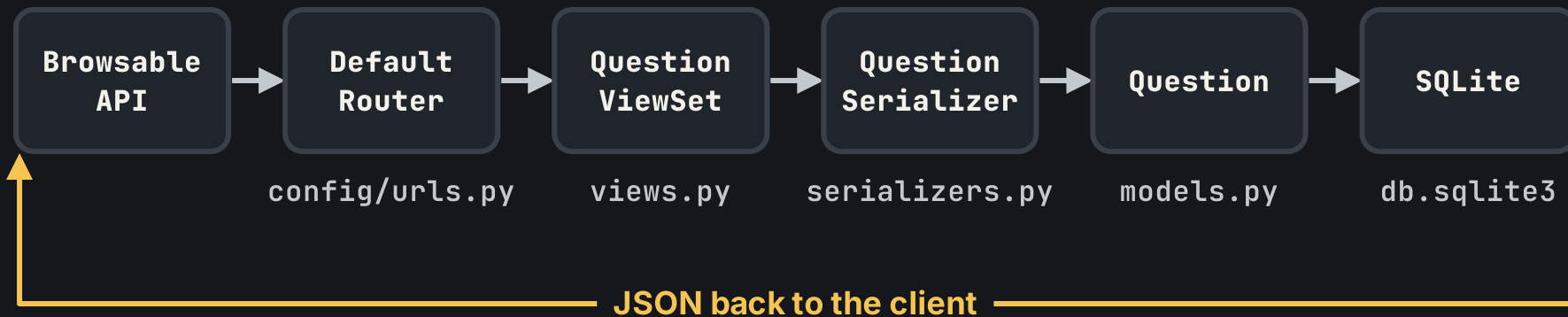
log in as a second user and upvote

THEN

fix ordering: ?ordering=-num_votes

What you built

Same map, real names. You wrote every one of these.



Where to go next

- Token auth (JWT)
- PostgreSQL instead of SQLite
- Tests with `APITestCase`
- Pagination
- Deploying your API
- A frontend that calls your API
- The official DRF tutorial

QR appears when
`{{RESOURCES_URL}}`
is set

All the links

Your questions

• • •

{{LIVE_BOARD_URL}}

SCREENSHOT

Django REST framework

Api Root / Question List

Question List

 Filters

OPTIONS

GET ▼

GET /api/questions/?ordering=-votes

HTTP 200 OK

Allow: GET, POST, HEAD, OPTIONS

Content-Type: application/json

Vary: Accept

[
 {
 "id": 1,
 "text": "Is DRF used in industry?",
 },
]

Developer Club wordmark · {{CLUB_LOGO}}

THANKS FOR BUILDING WITH US

NEXT DEVELOPER CLUB EVENT

{{NEXT_EVENT_TITLE}}

{{NEXT_EVENT_DATE}} · {{NEXT_EVENT_ROOM}}

STARTER REPO

github.com/joshualent/questions-api



Starter repo

tier-1-done

tier-2-auth

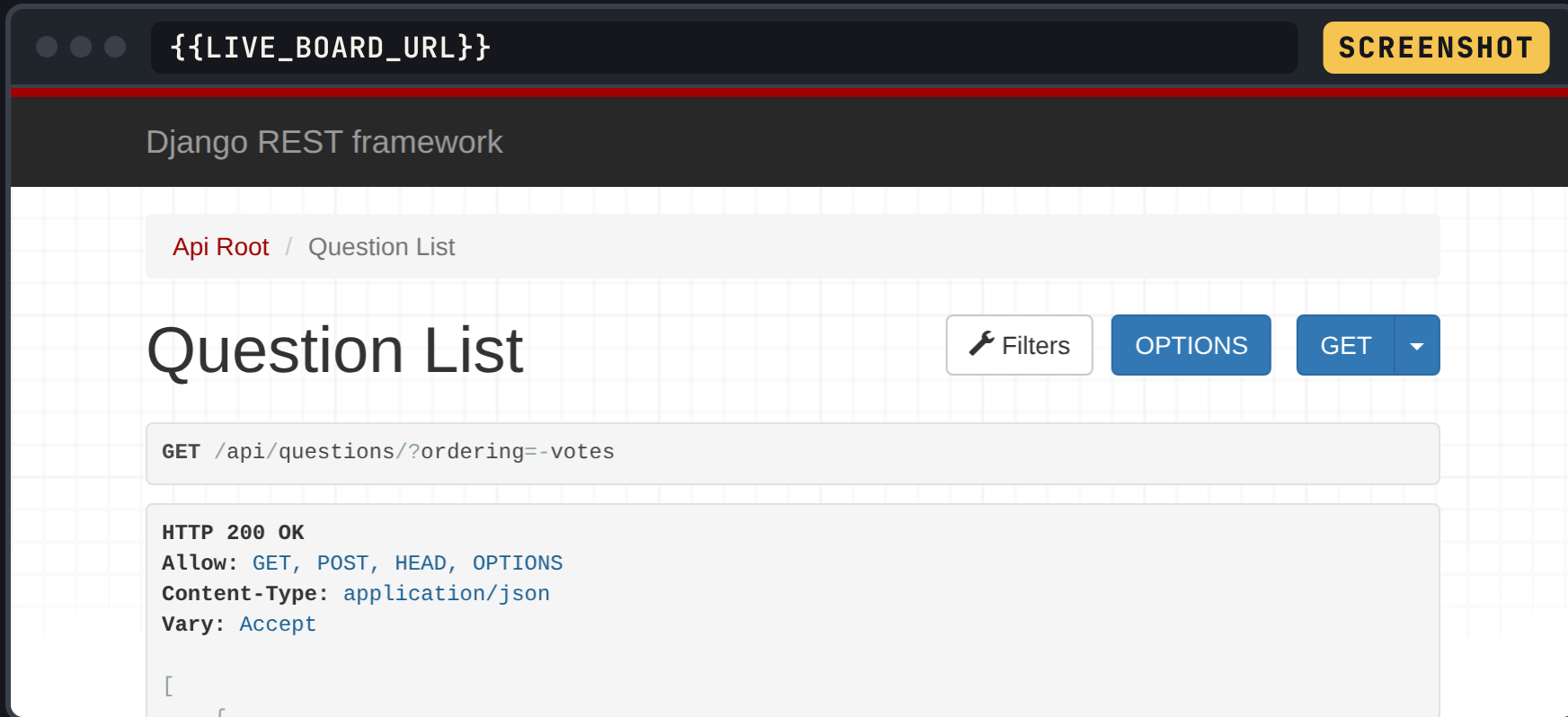
tier-3-m2m

Backstage

- 38** Fallback: Here's what you'll build (slide 2)
- 39** Fallback: Vote now (slide 25)
- 40** Fallback: Your questions (slide 34)
- 41** Clip still: server-rendered vs. API (slide 7)
- 42** Clip still: class to table (slide 14)
- 43** Clip still: counter to many-to-many (slide 28)
- 44** Reference v1: models.py
- 45** Reference v1: serializers.py
- 46** Reference v1: views.py (1/2)
- 47** Reference v1: views.py (2/2)
- 48** Reference v1: config/urls.py
- 49** Reference v2 Part A: login + permissions
- 50** Reference v2 Part B: models.py
- 51** Reference v2 Part B: serializers.py (1/2)
- 52** Reference v2 Part B: serializers.py (2/2)
- 53** Reference v2 Part B: views.py (1/3)
- 54** Reference v2 Part B: views.py (2/3)
- 55** Reference v2 Part B: views.py (3/3)
- 56** Troubleshooting: setup
- 57** Troubleshooting: database
- 58** Projector check
- 59** Title direction B: title + terminal
- 60** Title direction C: request line

Here's what you'll build

The live board: your questions, sorted by votes.



Vote now

Upvote what you want answered at the end.

Browser address bar: `{{LIVE_BOARD_URL}}` **SCREENSHOT**

Django REST framework

Api Root / Question List

Question List

Filters OPTIONS GET

GET /api/questions/?ordering=-votes

HTTP 200 OK
Allow: GET, POST, HEAD, OPTIONS
Content-Type: application/json
Vary: Accept

```
[
  {
    "id": 1,
    "text": "Is DRF used in industry?"
  }
]
```

QR appears
when
`{{LIVE_BOARD_URL}}`
is set

Scan to vote

Your questions

• • •

{{LIVE_BOARD_URL}}

SCREENSHOT

Django REST framework

Api Root / Question List

Question List

 Filters

OPTIONS

GET ▼

GET /api/questions/?ordering=-votes

HTTP 200 OK

Allow: GET, POST, HEAD, OPTIONS

Content-Type: application/json

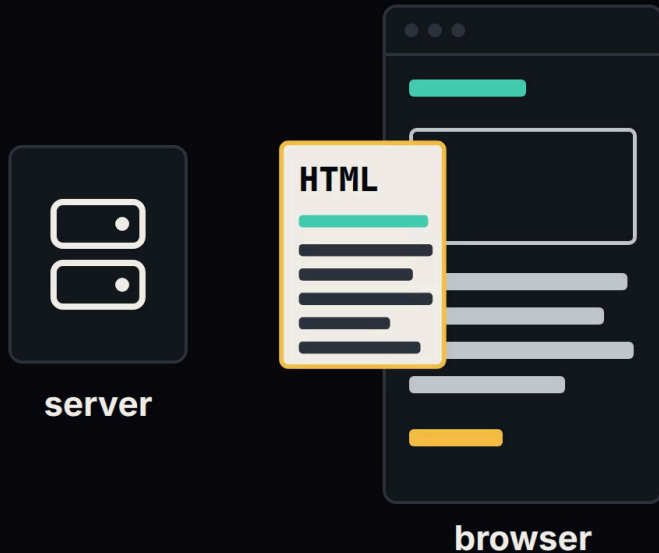
Vary: Accept

```
[
  {
    "id": 1,
    "text": "Is DRF used in industry?"
```

Server-rendered pages vs. an API

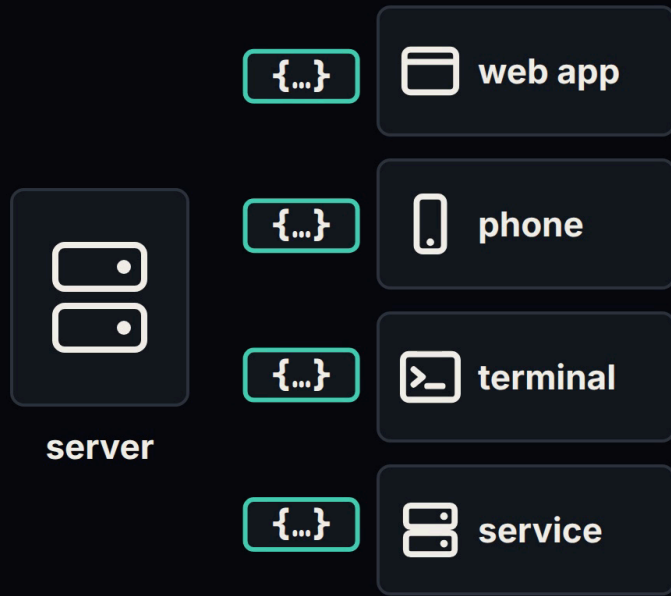
Server-rendered

whole page → one browser



API

small JSON → any client



Step 1: the model

```
class Question(models.Model):  
    text = models.CharField(max_length=300)  
    votes = models.IntegerField(default=0)  
    answered = models.BooleanField(default=False)  
    created_at = models.DateTimeField(auto_now_add=True)
```

questions_question

id	text	votes	answered	created_at
1	Is REST hard?	3	False	19:02
2	Do I need SQL?	7	False	19:05
3	How do I deploy?	4	True	19:09

An object is a row

A counter can't remember who voted

questions

id	text
1	Is REST hard?
2	Do I need SQL?

questions_question_voters

question_id	user_id
1	1
1	2
1	3
2	1

users

id	username
1	 sam
2	 ana
3	 lee

Django builds the join table

`questions/models.py`

```
from django.db import models

class Question(models.Model):
    text = models.CharField(max_length=300)
    votes = models.IntegerField(default=0)
    answered = models.BooleanField(default=False)
    created_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return self.text
```

```
questions/serializers.py
```

```
from rest_framework import serializers
from .models import Question

class QuestionSerializer(serializers.ModelSerializer):
    class Meta:
        model = Question
        fields = '__all__'
        read_only_fields = ['votes', 'answered']
```

questions/views.py

```
from rest_framework import viewsets
from rest_framework.decorators import action
from rest_framework.response import Response
from rest_framework.filters import OrderingFilter
from .models import Question
from .serializers import QuestionSerializer

class QuestionViewSet(viewsets.ModelViewSet):
    queryset = Question.objects.all()
    serializer_class = QuestionSerializer
    filter_backends = [OrderingFilter]
    ordering_fields = ['votes', 'created_at']
```

questions/views.py

```
@action(detail=True, methods=['post'])
def upvote(self, request, pk=None):
    question = self.get_object()
    question.votes += 1
    question.save()
    return Response(self.get_serializer(question).data)
```

```
@action(detail=True, methods=['post'])
def mark_answered(self, request, pk=None):
    question = self.get_object()
    question.answered = True
    question.save()
    return Response(self.get_serializer(question).data)
```

config/urls.py

```
from django.contrib import admin
from django.urls import path, include
from rest_framework.routers import DefaultRouter
from questions.views import QuestionViewSet

router = DefaultRouter()
router.register('questions', QuestionViewSet)

urlpatterns = [
    path('admin/', admin.site.urls),
    path('api/', include(router.urls)),
]
```

REFERENCE · V2 PART A

config/urls.py: add to urlpatterns

```
path('api-auth/', include('rest_framework.urls')),
```

config/settings.py: uncomment

```
REST_FRAMEWORK = {  
    "DEFAULT_PERMISSION_CLASSES": [  
        "rest_framework.permissions.IsAuthenticatedOrReadOnly",  
    ],  
}
```

questions/views.py: optional, admin-only mark_answered

```
from rest_framework.permissions import IsAdminUser  
  
@action(detail=True, methods=['post'], permission_classes=[IsAdminUser])  
def mark_answered(self, request, pk=None):  
    ...
```

`questions/models.py`

```
from django.contrib.auth.models import User
from django.db import models

class Question(models.Model):
    text = models.CharField(max_length=300)
    voters = models.ManyToManyField(User, related_name='voted_questions', blank=True)
    answered = models.BooleanField(default=False)
    created_at = models.DateTimeField(auto_now_add=True)

    @property
    def vote_count(self):
        return self.voters.count()

    def __str__(self):
        return self.text
```

questions/serializers.py

```
class QuestionSerializer(serializers.ModelSerializer):
    vote_count = serializers.ReadOnlyField()
    has_voted = serializers.SerializerMethodField()

    class Meta:
        model = Question
        fields = ['id', 'text', 'vote_count', 'has_voted', 'answered', 'created_at']
        read_only_fields = ['answered']
```

questions/serializers.py

```
def get_has_voted(self, obj):  
    request = self.context.get('request')  
    return bool(  
        request  
        and request.user.is_authenticated  
        and obj.voters.filter(pk=request.user.pk).exists()  
    )
```

questions/views.py

```
from django.db.models import Count
from rest_framework import status
from rest_framework.permissions import IsAuthenticated, IsAdminUser

class QuestionViewSet(viewsets.ModelViewSet):
    queryset = Question.objects.annotate(num_votes=Count('voters'))
    serializer_class = QuestionSerializer
    filter_backends = [OrderingFilter]
    ordering_fields = ['num_votes', 'created_at']
```

questions/views.py

```
@action(detail=True, methods=['post'], permission_classes=[IsAuthenticated])
def upvote(self, request, pk=None):
    question = self.get_object()
    if question.voters.filter(pk=request.user.pk).exists():
        return Response(
            {'detail': 'You already voted for this question.'},
            status=status.HTTP_400_BAD_REQUEST,
        )
    question.voters.add(request.user)
    return Response(self.get_serializer(question).data)
```

questions/views.py

```
@action(detail=True, methods=['post'], permission_classes=[IsAdminUser])
def mark_answered(self, request, pk=None):
    question = self.get_object()
    question.answered = True
    question.save()
    return Response(self.get_serializer(question).data)
```

Troubleshooting: setup

uv: command not found

macOS / Linux, then open a new terminal:

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

Windows (PowerShell), then open a new terminal:

```
powershell -ExecutionPolicy Bypass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

Error: That port is already in use.

```
uv run python manage.py runserver 8001
```

...and use `127.0.0.1:8001` in the browser.

Troubleshooting: database

no such table: questions_question

The migration was never made or applied:

```
uv run python manage.py makemigrations
uv run python manage.py migrate
```

no such column after a checkpoint checkout

Your database is from different code. Start it fresh:

```
rm db.sqlite3          # Windows: del db.sqlite3
uv run python manage.py migrate
```

No changes detected

Save models.py first, then run makemigrations again.

Projector check

	--bg	#15171c
	--surface	#20242b
	--border	#3a404a
	--text	#f4f1ea
	--text-muted	#c4c8cf
	--accent	#4fd1b5
	--accent-2	#f5c451
	--get	#6cb6ff
	--post	#5fd68a
	--patch	#f5c451
	--delete	#ff7a7a
	--ok	#5fd68a
	--error	#ff7a7a

Title 40 px

Body 24 px: can the back row read this?

code 18 px: `queryset = Question.objects.all()`

Diagram label 16 px (the smallest text in the deck)

Muted text 24 px (--text-muted)

GET

POST

PATCH

DELETE

201 Created

403 Forbidden

Developer Club wordmark · {{CLUB_LOGO}}

BUILD A Q&A API

Python · Django · DRF

Mon, Oct 19 · Floyd Hall D-109

```
$ curl .../questions/?ordering=-votes  
  
[  
  {  
    "id": 4,  
    "text": "Do I need SQL?",  
    "votes": 9,  
    "answered": false  
  },  
  ...  
]
```

Direction B

Developer Club wordmark · {{CLUB_LOGO}}

POST

/api/questions/

201 Created

BUILD A Q&A API

with Python, Django & Django REST Framework

Monday, Oct 19 · Floyd Hall D-109

Direction C